

TCC · DOCUMENTO DE BANCO DE DADOS

ControlaAI TCC — Banco de Dados PostgreSQL (Colunas, Tabelas, Relações e Chaves)

Como o PostgreSQL sustenta o Controla.AI: cadastro, WhatsApp, gastos, metas — mais **arquitetura visual**, **modelagem**, **conexões FK** e **dicionário de dados** (10 páginas).

ESTRUTURA DO DOCUMENTO

Págs. 2–5 — 3 tópicos (papel · Railway · tabelas/relações)

Pág. 6 — Arquitetura (diagrama ER)

Pág. 7 — Modelagem + detalhes visuais

Pág. 8 — Conexões entre tabelas (18 FK)

Págs. 9–10 — Dicionário de dados

EQUIPE

Davi Almeida
Leonardo Sena
Gustavo Biscoto

Agosto / 2026
Curitiba — PR

1 O banco no Controla.AI

PostgreSQL como memória única do produto

O **Controla.AI** é um assistente financeiro em que o usuário lança gastos por **WhatsApp** (“gastei 45 no almoço”), pelo painel na **Vercel** (**controlaai-frontend.vercel.app**) ou por **PDF** de extrato. O **PostgreSQL 15+** na **Railway** guarda tudo: quem é a pessoa (**users**), o que gastou (**transactions**), a categoria (**categories**) e a meta (**goals**). No Zap, o backend acha o telefone em **users.phone**, a **OpenAI** interpreta valor e categoria, grava o lançamento e liga a mensagem em **whatsapp_messages**. No site, o dashboard só lê o banco — saldo, gráficos e alertas vêm das mesmas linhas.

A modelagem tem **16 tabelas** e **18 relações**, todas em torno de **users** (hub central). Conta: **users**, **user_settings**, **user_consents** (LGPD com versão, IP e data). Financeiro: **transactions**, **categories**, **budgets**, **recurring_transactions**. Metas: **goals** e **goal_checkpoints** (alerta 80%/100%). Canais: **whatsapp_messages**, **ai_logs**, **document_imports**, **subscriptions** (**Stripe**). Cada rota da API filtra pelo **user_id** do **JWT** — uma conta nunca vê o financeiro de outra.

O **PostgreSQL** aplica regras sozinho, sem depender do front. **UNIQUE** em **users.email** impede dois cadastros com o mesmo e-mail. **UUID** gera id único por linha. **CASCADE**: ao apagar um usuário, o Postgres remove gastos, metas e sessões ligados. **SET NULL**: ao apagar categoria personalizada, o lançamento permanece. **Enums** fixam plano, tipo e origem. A base já tem dados reais: **admin@admin.com**, 210+ lançamentos do Leonardo e 5 metas com checkpoints.

16

tabelas

18

relações FK

4

módulos visuais

1

hub: users

Conta e LGPD

users · **user_settings** · **user_consents**

Financeiro

transactions · **categories** · **budgets** · **recurring**

Metas

goals · **goal_checkpoints** — progresso e alertas

Canais

whatsapp_messages · **ai_logs** · **document_imports** · **subscriptions**

Síntese do Tópico 1

O **PostgreSQL** na **Railway** une WhatsApp, site e PDF numa só memória. Diagramas oficiais: pasta **PNGs modelagens banco dados**.

2

Onde o banco do Controla.AI roda

Railway · PostgreSQL · Redis · DBeaver · Vercel

Em produção, o **PostgreSQL** fica na **Railway** junto com a API **Node/Fastify**. O site **React** fica na **Vercel** e consome essa API. A equipe administra e demonstra o banco pelo **DBeaver**, conectado à mesma base com **SSL**. A Railway usa a extensão **pgcrypto** para **UUID**; a API fala com o banco na rede interna; para DBeaver e scripts existe URL pública com **SSL**. O **Redis** acelera sessões e KPIs — a verdade dos dados continua no Postgres. Frontend: **controlaai-frontend.vercel.app** · API: **backend-production-c328.up.railway.app** · health em **/health**.

No cadastro, o backend grava o usuário, cria preferências em **user_settings** e registra três linhas em **user_consent**s (termos, privacidade e **LGPD**) com versão, horário, IP e navegador — tudo em uma única transação. A senha só existe como hash **bcrypt** em **users.password_hash**. No login, o sistema busca o e-mail, compara o hash e emite um **JWT** de 7 dias. Cada rota protegida filtra pelo **user_id** do token. Conta admin (**admin@admin.com**) libera painel operacional; planos pagos atualizam **users.plan** via webhook **Stripe** em **subscriptions**.

O **DBeaver Community** valida seeds, confere **ai_logs** e **whatsapp_messages** e executa o script de recriação (**backend/scripts/novo-banco-railway-COMPLETO.sql**). O código em **github.com/visaobusinesstech/CONTROLAAIGastosDeploy** dispara frontend na Vercel e backend na Railway. Variáveis sensíveis (**DATABASE_URL**, **JWT_SECRET**) ficam só no painel da Railway. Migrations em **drizzle/** acompanham o schema.

Camada	Tecnologia	Responsabilidade
Painel e login	React · Vite · Vercel	Dashboard, metas, chat, cadastro
API	Node · Fastify · Railway	Auth, lançamentos, WhatsApp, Stripe
Persistência	PostgreSQL · Drizzle	16 tabelas, regras e histórico
Cache	Redis (Railway)	Acelerar sessões e KPIs
IA	GPT-4o-mini · Whisper	Parser de mensagem e áudio
WhatsApp	Baileys	Canal oficial de entrada
Billing	Stripe	Planos Pro/Premium no banco
Console	DBeaver	Inspeção e demo do schema/dados

Produção atual

Frontend: **controlaai-frontend.vercel.app** · API: **backend-production-c328.up.railway.app** · Login: **admin@admin.com** / 123456 · Banco: **PostgreSQL Railway**.

3

Estrutura interna do banco

Tabelas · colunas · uso no produto

A tabela **users** segura o sistema. Colunas centrais: **id**, **name**, **email** (único), **password_hash**, **phone** (WhatsApp), **plan** (free/pro/premium), **stripe_customer_id**, **trial_ends_at** e **billing_grandfathered**. O login busca por email; o agente Zap resolve pelo **phone**; o billing atualiza **plan** quando o **Stripe** confirma pagamento.

Em **transactions**: **amount**, **type** (expense/income), **occurred_at**, **source** (whatsapp/web/manual/recurring/pdf), **raw_message**, **user_id** e **category_id**. Em **categories**: nome, ícone, cor e se é padrão do sistema (**user_id** nulo) ou do usuário. O histórico sobrevive se a categoria personalizada for removida (**SET NULL**).

goals guarda meta de limite ou poupança, valor alvo, prazo e categoria opcional. **goal_checkpoints** guarda o percentual do mês e se o alerta 80%/100% já saiu. **budgets** amarra renda esperada e teto de despesa por usuário/mês. **recurring_transactions** descreve aluguel, academia e assinaturas; um job materializa essas linhas como **transactions** na data.

Colunas que amarram produto e banco: **users.email** + **password_hash** (login), **users.phone** (Zap), **transactions.amount/type/source**, **user_id/category_id**, **goals** + **goal_checkpoints**, **user_consents.document_version**, **ai_logs.cost_usd** e **whatsapp_messages.transaction_id**.

Conta users · user_settings · user_consents	Financeiro transactions · categories · budgets · recurring
Metas goals · goal_checkpoints	Canais whatsapp_messages · ai_logs · document_imports · subscriptions
Resumo Login · telefone Zap · valor/tipo/origem · dono e categoria · metas com checkpoint · versão LGPD · custo da IA · vínculo mensagem→gasto. Detalhamento nas págs. 6–10.	

Há **18 relações**. **users** é o hub: **user_settings**, **transactions**, **categories**, **goals**, **budgets**, **ai_conversations**, **subscriptions**, **user_consent**s e **whatsapp_sessions** apontam para **users.id**. **transactions** e **goals** apontam para **categories**. **goal_checkpoints** → **goals**. **whatsapp_messages** → **users** e, quando gera gasto, → **transactions**. Deleção de usuário: **CASCADE**. Deleção de categoria: **SET NULL**.

Ligação	O que garante
transactions → users	Todo gasto tem dono; dashboard e Zap usam o mesmo vínculo
transactions → categories	Gráficos e metas por tipo de gasto
goals → users	Meta pessoal; alerta na categoria
whatsapp_messages → transactions	Rastreio mensagem → lançamento
user_consents → users	Auditoria LGPD do cadastro
subscriptions → users	Plano pago em users.plan

Fluxo **WhatsApp** → **banco**: 1) **Baileys** grava **whatsapp_messages**. 2) Resolve **users.phone**. 3) **OpenAI** extrai valor/tipo/categoria; **ai_logs** registra custo. 4) **INSERT** em **transactions**. 5) Mensagem recebe o id do lançamento. 6) Meta recalcula percentual e checkpoint.

A base já tem **8 usuários**, **210+** lançamentos do Leonardo (mar–ago/2026), **5 metas** com checkpoints, orçamentos, categorias, amostras de Zap e logs de IA — alimentando dashboard e consultas no **DBeaver**.

Conta	E-mail	Senha	Uso
Admin	admin@admin.com	123456	Premium · painel admin
Leonardo	leonardo.sena@unicesumar.edu.br	123456	+210 lançamentos · demo
Davi / Gustavo	*@unicesumar.edu.br	123456	Equipe TCC · premium
Demos	marina / carlos / juliana / roberto	123456	Planos free e pro

Próximas páginas

Pág. 6 Arquitetura com os 4 grupos coloridos · **Pág. 7** Modelagem (PK, FK, CASCADE) · **Pág. 8** Tabela das 18 conexões · **Págs. 9–10** Dicionário: o que cada tabela guarda.



Arquitetura do banco

Como as tabelas se organizam no PostgreSQL do Controla.AI

Este desenho é o **mapa do banco**. Cada caixa é uma **tabela** (um tipo de informação guardada). As cores separam o banco em **4 partes**: conta do usuário, dinheiro, metas e inteligência artificial. Quase todas as caixas apontam para **users** — a tabela do cadastro. Isso significa: todo gasto, toda meta e toda mensagem do WhatsApp pertencem a **alguém**. As setas A–F destacam ligações importantes (settings, categoria, meta e vínculo mensagem → lançamento).

Azul — Usuário e WhatsApp

Guarda **quem é a pessoa** (**users**: nome, e-mail, senha, telefone), as preferências (**user_settings**) e a conexão do Zap (**whatsapp_connection** / **whatsapp_sessions**). É o ponto de partida de tudo.

Teal — Metas

goals é o limite ou a poupança que a pessoa definiu (ex.: gastar no máx. R\$ 500 em delivery). **goal_checkpoints** guarda, mês a mês, quanto já usou e se o alerta de 80% ou 100% já foi enviado.

Verde — Núcleo financeiro

Guarda o dinheiro de verdade: cada gasto/receita em **transactions**, o tipo em **categories** (ex.: Alimentação), orçamento em **budgets**, contas fixas em **recurring_transactions** e o histórico das mensagens em **whatsapp_messages**.

Roxo — IA e assinatura

Histórico do chat (**ai_conversations**), custo de cada chamada OpenAI (**ai_logs**), hábitos aprendidos (**financial_memory**), importação de PDF (**document_imports**) e plano pago Stripe (**subscriptions**).

Arquitetura do banco — tabelas e relacionamentos

16 tabelas · 6 setas A-F (sem cruzamento) · users.id na caixa azul = tabela FK abaixo

PK: FK: A-F = setas: A: user_settings, B: D: category_id, E: goal_id, F: transaction_id

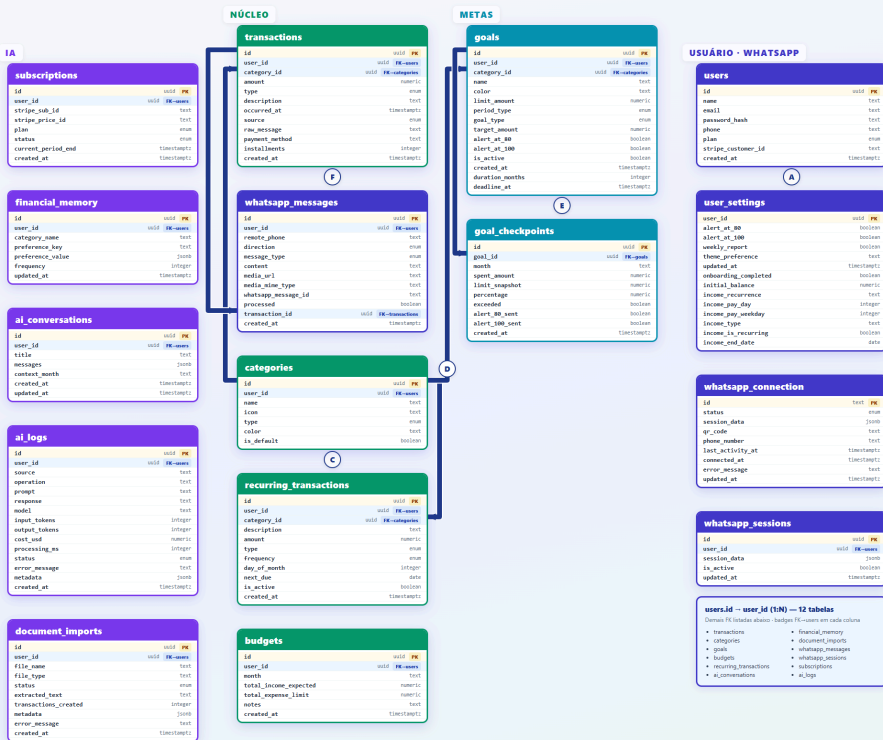


Diagrama oficial · pasta PNGs modelagens banco dados · amarelo = chave primária (PK)



Modelagem de dados

O que a modelagem decide — e o que o desenho abaixo prova

Modelar é decidir: quais tabelas existem, quais colunas cada uma tem e como uma tabela aponta para outra. No Controla.AI a modelagem segue uma regra simples: **um usuário é o dono** dos gastos, das metas e das mensagens. A imagem abaixo mostra o mesmo banco de forma resumida e, embaixo, a **lista das 18 ligações** (chaves estrangeiras) com o que acontece ao apagar um registro. **1:1** = um usuário tem um único user_settings; **1:N** = um usuário tem muitos gastos.

Chave primária (PK)

É o “RG” da linha. Em quase todas as tabelas é o campo **id** (UUID). Duas linhas nunca têm o mesmo id. No diagrama aparece em destaque amarelo.

CASCADE (apagar em cascata)

Se apagarmos o usuário, o Postgres apaga também os gastos, metas e preferências dele. Não sobra lixo órfão no banco.

Chave estrangeira (FK)

É um campo que **aponta** para o id de outra tabela. Ex.: **transactions.user_id** aponta para **users.id** — assim sabemos de quem é o gasto.

SET NULL (só limpa o vínculo)

Se apagarmos uma categoria, o gasto **permanece**; só o campo **category_id** fica vazio. O histórico do dashboard não some.

Arquitetura completa — railway

Controla.AI · Sun Jun 07 2026 20:58:42 GMT-0300 (Horário Padrão de Brasília) · 16 tabelas · todas as colunas · 18 FK

Diagrama — linhas curtas entre tabelas vizinhas

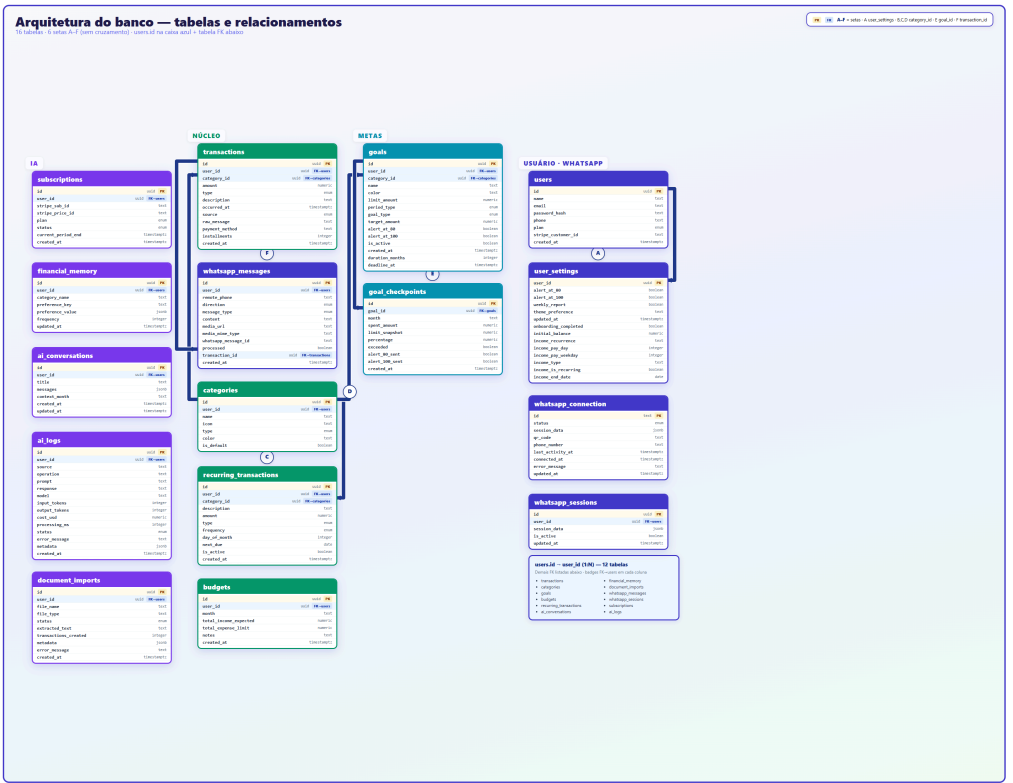


Tabela de relacionamentos (FK)

#	Origem (FK)	Destino (FK)	Card.	Do delete
1	users.id	user_settings.user_id	1:1	CASCADE
2	users.id	transactions.user_id	1:N	CASCADE
3	users.id	categories.user_id	1:N	CASCADE
4	users.id	goals.user_id	1:N	CASCADE
5	users.id	budgets.user_id	1:N	CASCADE
6	users.id	recurring_transactions.user_id	1:N	CASCADE
7	users.id	ai_conversations.user_id	1:N	CASCADE
8	users.id	financial_memory.user_id	1:N	CASCADE
9	users.id	document_imports.user_id	1:N	CASCADE
10	users.id	whatsapp_messages.user_id	1:N	SET NULL
11	users.id	whatsapp_sessions.user_id	1:N	CASCADE
12	users.id	subscriptions.user_id	1:N	CASCADE
13	users.id	ai_logs.user_id	1:N	SET NULL
14	categories.id	transactions.category_id	1:N	SET NULL
15	categories.id	goals.category_id	1:N	SET NULL
16	categories.id	recurring_transactions.category_id	1:N	SET NULL
17	goals.id	goal_checkpoints.goal_id	1:N	CASCADE
18	transactions.id	whatsapp_messages.transaction_id	0:1	SET NULL

Mesma modelagem · diagrama + tabela das 18 FK (cardinalidade e ON DELETE)

C

Conexões entre tabelas

Relação entre os dados — com exemplo do produto

A tabela abaixo lista as **18 chaves estrangeiras** do banco. Leia assim: a coluna **Origem** é o “pai”; a coluna **Destino** é o campo filho que guarda o id do pai. **Card.** diz quantos filhos um pai pode ter. **On delete** diz o que o Postgres faz se o pai for apagado.

Exemplo no produto

Leonardo manda no WhatsApp: “gastei 45 no almoço”. O sistema acha o Leonardo pelo telefone em **users** → grava a mensagem em **whatsapp_messages** (com **user_id**) → cria o gasto em **transactions** (mesmo **user_id**, categoria Alimentação) → preenche **whatsapp_messages.transaction_id** apontando para esse gasto. Se Leonardo tiver meta de Alimentação, atualiza **goal_checkpoints**.

- 1:1 — um para um

Cada usuário tem **no máximo uma** linha em user_settings (preferências).
- 1:N — um para muitos

Um usuário tem **vários** gastos, várias metas, várias mensagens.
- 0:1 — opcional

Uma mensagem do Zap **pode** gerar um gasto (ou não — se for só “oi”).
- Tabela isolada

whatsapp_connection é a conexão do número oficial do app — não depende de um usuário.

#	Origem (PK)	Destino (FK)	Card.	On delete
1	users.id	user_settings.user_id	1:1	CASCADE
2	users.id	transactions.user_id	1:N	CASCADE
3	users.id	categories.user_id	1:N	CASCADE
4	users.id	goals.user_id	1:N	CASCADE
5	users.id	budgets.user_id	1:N	CASCADE
6	users.id	recurring_transactions.user_id	1:N	CASCADE
7	users.id	ai_conversations.user_id	1:N	CASCADE
8	users.id	financial_memory.user_id	1:N	CASCADE
9	users.id	document_imports.user_id	1:N	CASCADE
10	users.id	whatsapp_messages.user_id	1:N	SET NULL
11	users.id	whatsapp_sessions.user_id	1:N	CASCADE
12	users.id	subscriptions.user_id	1:N	CASCADE
13	users.id	ai_logs.user_id	1:N	SET NULL
14	categories.id	transactions.category_id	1:N	SET NULL
15	categories.id	goals.category_id	1:N	SET NULL
16	categories.id	recurring_transactions.category_id	1:N	SET NULL
17	goals.id	goal_checkpoints.goal_id	1:N	CASCADE
18	transactions.id	whatsapp_messages.transaction_id	0:1	SET NULL

Resumo

Das 18 ligações, **13 saem de users.id**. Categorias ligam em 3 lugares (gastos, metas, recorrentes). A ligação 18 fecha o ciclo Zap → gasto.
Fonte: **CONEXOES_BANCO_DADOS.md**.

Dicionário de dados — Conta · Dinheiro · Metas

Para cada tabela: para que serve + colunas principais

Dicionário de dados = lista do que cada tabela guarda e o que cada coluna significa. Aqui estão as 8 tabelas do dia a dia financeiro. Na próxima página: IA, WhatsApp e assinatura.

users conta da pessoa		
Cadastro: nome, e-mail (login), senha criptografada, telefone do Zap e plano (free/pro/premium).		
Coluna	Tipo	Chave
id	uuid	PK
name	text	—
email	text UNIQUE	—
password_hash	text	—
phone	text	—
plan	enum	—
stripe_customer_id	text	—
created_at	timestamptz	—

Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
name	text	—
icon / color	text	—
type	enum	—
is_default	boolean	—

budgets orçamento do mês		
Renda esperada e teto de gastos daquele mês (YYYY-MM) para aquele usuário.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
month	text YYYY-MM	—
total_income_expected	numeric	—
total_expense_limit	numeric	—
notes	text	—

goals meta financeira		
Limite de gasto ou meta de poupança, com prazo e alertas em 80% e 100%.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
category_id	uuid	FK → categories
name / color	text	—
limit_amount / target	numeric	—
period_type / goal_type	enum	—
duration_months / deadline	int / ts	—
alert_at_80/100 · is_active	boolean	—

user_settings preferências		
Uma linha por usuário: alertas de meta, tema, onboarding e dados de renda (dia do salário etc.).		
Coluna	Tipo	Chave
user_id	uuid	PK/FK
alert_at_80/100	boolean	—
weekly_report	boolean	—
theme_preference	text	—
onboarding_completed	boolean	—
initial_balance	numeric	—
income_*	text/int/date	—
updated_at	timestampz	—

transactions cada gasto/receita		
O coração do app. Valor, data, origem (WhatsApp/web/PDF) e a mensagem original quando veio do Zap.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
category_id	uuid	FK → categories
amount / type	numeric / enum	—
occurred_at	timestamptz	—
source / raw_message	text	—
payment_method	text	—
installments	integer	—

recurring_transactions		
contas fixas		
Aluguel, academia, Netflix: o sistema cria o lançamento automaticamente na data.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
category_id	uuid	FK → categories
description / amount	text / numeric	—
frequency	enum	—
day_of_month / next_due	int / date	—
is_active	boolean	—

goal_checkpoints progresso da meta		
Foto do mês: quanto já gastou, % usado e se o alerta já saiu no WhatsApp.		
Coluna	Tipo	Chave
id	uuid	PK
goal_id	uuid	FK → goals
month	text	—
spent_amount / limit_snapshot	numeric	—
percentage	numeric	—
exceeded	boolean	—
alert_80/100_sent	boolean	—



Dicionário de dados — IA · WhatsApp · Assinatura

Tabelas de canal, inteligência artificial e billing

Estas 8 tabelas sustentam o **canal WhatsApp**, o **chat com IA**, a **importação de PDF** e o **pagamento Stripe**. Junto com a página anterior, cobrem as **16 tabelas** do PostgreSQL.

ai_conversations chat com a IA		
Histórico do chat no site (perguntas e respostas em JSON).		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
title	text	—
messages	jsonb	—
context_month	text	—
created_at / updated_at	timestamptz	—

financial_memory hábitos aprendidos		
A IA lembra preferências (ex.: “almoço” costuma ser Alimentação).		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
category_name	text	—
preference_key	text	—
preference_value	jsonb	—
frequency	integer	—

subscriptions plano pago		
Assinatura Stripe: id da cobrança, plano e até quando está válida.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
stripe_sub_id / price_id	text	—
plan / status	enum	—
current_period_end	timestamptz	—

whatsapp_sessions sessão por usuário		
Dados técnicos da sessão Baileys ligados a um usuário (quando aplicável).		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
session_data	jsonb	—
is_active	boolean	—
updated_at	timestamptz	—

ai_logs custo da OpenAI		
Cada chamada à IA: modelo, tokens, tempo e custo em dólar — para auditar gasto.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
source / operation	text	—
prompt / response / model	text	—
input/output_tokens	integer	—
cost_usd / processing_ms	numeric / int	—
status / error_message	enum / text	—
metadata	jsonb	—

document_imports PDF / extrato		
Quando o usuário sobe um extrato: nome do arquivo, status e quantos lançamentos nasceram.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
file_name / file_type	text	—
status	enum	—
extracted_text	text	—
transactions_created	integer	—
metadata / error_message	jsonb / text	—

whatsapp_messages mensagens do Zap		
Tudo que entra e sai no WhatsApp. Se virar gasto, transaction_id aponta para o lançamento.		
Coluna	Tipo	Chave
id	uuid	PK
user_id	uuid	FK → users
remote_phone	text	—
direction / message_type	enum	—
content / media_*	text	—
processed	boolean	—
transaction_id	uuid	FK → transactions

whatsapp_connection número oficial do app		
Única conexão do bot (QR code, status conectado/desconectado). Não tem user_id.		
Coluna	Tipo	Chave
id	text (=main)	PK
status	enum	—
session_data / qr_code	jsonb / text	—
phone_number	text	—
last_activity_at / connected_at	timestamptz	—
error_message	text	—

Síntese

O Controla.AI tem **16 tabelas** e **18 ligações**. O centro é **users**. O dinheiro fica em **transactions**. O Zap liga mensagem ao gasto. A IA registra custo em **ai_logs**. Metas medem progresso em **goal_checkpoints**. Tudo isso roda no **PostgreSQL da Railway** e aparece no dashboard e no DBEaver.